

E-Commerce order management system

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

AULA VENKATA NAVEEN– 25B11AIA075

NAKKA KARNTHI – 25B11AI606

NEETIPALLI SATYA SAI HANUMA PHANINDRA – 25B11AI839

SATYA NANDA KISHORE GOPI – 25B11AIA79

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Ecommerce order management system” is being submitted by 25B11AI075 , 25B11AI606, 25B11AI839, 25B11AIA79 in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)

Associate professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.

Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Ecommerce order management system” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu , Department of AIML, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

Avula Venkata Naveen (25B11AI075)

Nakka Kranthi (25B11AI606)

Neetipalli Satya Sai Hanuma Phanindra (25B11AI839)

Satya Nanda Kishore Gopi (25B11AIA79)

Date: _____

Place: Surampalem

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Submitted by:

S.No	Student Name	Roll Number
1	A VENKATA NAVEEN	25B11AI075
2	N KRANTHI	25B11AI606
3	N. PHANINDRA	25B11AI839
4	GSN KISHORE	25B11AIA79

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27 Date of Submission: 14 September 2026

GitHub Repository Link: <https://github.com/E-commerce-Order-management-system/Smart-E-Commerce-Order-Management-System>

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu ,, Department of AIML, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to K NAGABHARGAVI, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

Avula Venkata Naveen 25B11AI075

Nakka Kranthi 25B11AI606

Neetipalli Satya Sai Hanuma Phanindra 25B11AI839

Satya Nanda Kishore Gopi 25B11AIA79

ABSTRACT

The E-Commerce Order Management System is a PostgreSQL-based database project developed to provide a structured, efficient, and reliable solution for managing essential operations in an e-commerce environment. The system is designed to handle and organize information related to customers, products, categories, addresses, orders, order items, and payments while maintaining accurate relationships among these entities. By storing data in a well-structured relational database, the system helps reduce data redundancy, improve data consistency, and simplify the process of retrieving and managing e-commerce information.

The database follows the principles of relational database design and normalization and is normalized up to Third Normal Form (3NF). This ensures that data is logically organized and unnecessary duplication is minimized. Various integrity constraints, including primary keys, foreign keys, unique constraints, check constraints, and default values, are implemented to maintain data accuracy and consistency. These constraints help prevent invalid or inconsistent data from being inserted into the database and ensure that relationships between different entities are properly maintained.

The system makes extensive use of PostgreSQL and SQL features to perform different database operations. Basic CRUD operations (Create, Read, Update, and Delete) are implemented for managing records. Joins are used to retrieve related information from multiple tables, while aggregate functions support calculations and analysis of order and product data. Subqueries are used for retrieving information based on conditions involving other queries. In addition, views are created to simplify access to frequently required information and provide a convenient way of presenting related data.

To improve the automation and reliability of the database, the system also includes a stored procedure for performing predefined database operations and a stock-validation trigger that automatically checks product stock availability during relevant order operations. These features demonstrate how database-level programming can be used to enforce business rules and maintain data integrity without relying entirely on application-level logic.

Overall, the E-Commerce Order Management System demonstrates the practical implementation of database design, entity relationships, normalization, SQL queries, constraints, views, stored procedures, triggers, and data integrity mechanisms in a real-world e-commerce scenario. The project provides a strong foundation for understanding how relational database systems can be designed and implemented to efficiently manage interconnected business data and support reliable order management operations.

TABLE OF CONTENTS

TABLE OF CONTENTS	3
1. ABSTRACT	7
2. INTRODUCTION	7
1. Background	7
2. Purpose of the Database	7
3. PROBLEM STATEMENT	8
4. OBJECTIVES	8
5. TOOLS & TECHNOLOGIES USED	9
6. DATABASE DESIGN	10
1. Identified Entities	10
2. Entity Attributes	11
3. ER Diagram	13
4. Relationships & Cardinality	13
5. Relational Database Schema	15
6. Normalization	17
7. DATABASE IMPLEMENTATION	18
1. DDL — Database and Table Creation	18
2. Sample Data — DML	21
3. Constraints & Data Integrity	22
4. SQL Operations & Queries	22
5. Sample Outputs / Screenshots	24
8. MODULES & FUNCTIONALITIES	28
9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES	28
10. ENHANCEMENTS & FUTURE IMPROVEMENTS	29
11. CONCLUSION	29
12. REFERENCES	Error! Bookmark not defined.
APPENDIX	Error! Bookmark not defined.
A. Complete SQL Script	Error! Bookmark not defined.
B. Additional Screenshots / Outputs	Error! Bookmark not defined.

1. ABSTRACT

The **E-Commerce Order Management System** is a PostgreSQL-based database project designed to efficiently manage customers, products, orders, order items, and payments in an e-commerce environment. The system organizes data into related entities to reduce redundancy and maintain consistency. The database is normalized up to **Third Normal Form (3NF)** and uses primary keys, foreign keys, unique constraints, check constraints, and default values to ensure data integrity. PostgreSQL features such as **CRUD operations, joins, aggregate functions, subqueries, views, a stored procedure, and a stock-validation trigger** are implemented to support database operations and analysis. The system provides a structured approach to managing order information and demonstrates the practical application of database design, SQL, normalization, and integrity constraints in an e-commerce environment.

2. INTRODUCTION

2.1 Background

E-commerce platforms manage a large amount of interconnected information. A single customer may have multiple addresses and phone numbers, products belong to categories, and an order may contain multiple products. Payments are associated with orders and order items preserve the quantity and price applicable at the time of purchase. A relational database is required to maintain these relationships consistently and to support fast retrieval, updates, reporting, and transaction-oriented operations.

2.2 Purpose of the Database

The database is intended for customers and store administrators. It manages customer details, delivery addresses, product catalog information, stock quantities, orders, order items, payment records, and reporting information. The design separates repeating and multi-valued information into related tables and uses foreign keys to preserve referential integrity.

3. PROBLEM STATEMENT

A basic or manually maintained e-commerce order process can lead to duplicate records, inconsistent product information, incorrect stock handling, difficulty locating customer orders, and time-consuming sales calculations. The proposed E-Commerce Order Management System addresses these limitations through a normalized relational database with clear relationships, constraints, SQL queries, views, and controlled operations for order and payment management.

4. OBJECTIVES

- Objective 1: Design and implement a structured PostgreSQL database for managing customers, products, orders, order items, and payments.
- Objective 2: Normalize the database up to **Third Normal Form (3NF)** to reduce data redundancy and avoid insertion, update, and deletion anomalies.
- Objective 3: Implement **primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints** to maintain data integrity and consistency.
- Objective 4: Implement and demonstrate **CRUD operations, joins, aggregate functions, subqueries, GROUP BY/HAVING, and database views** for efficient data retrieval and analysis.
- Objective 5: Implement advanced PostgreSQL features such as a **stored procedure and stock-validation trigger** to automate and validate order-related operations.

5. TOOLS & TECHNOLOGIES USED



Category	Tool / Technology Used
Database Server	Supabase
Query / Admin Tool	Supabase
Front-end (if any)	Not applicable / SQL-based demonstration
Operating System	Windows
Version Control GitHub	GitHub repository

6. DATABASE DESIGN

6.1 Identified Entities

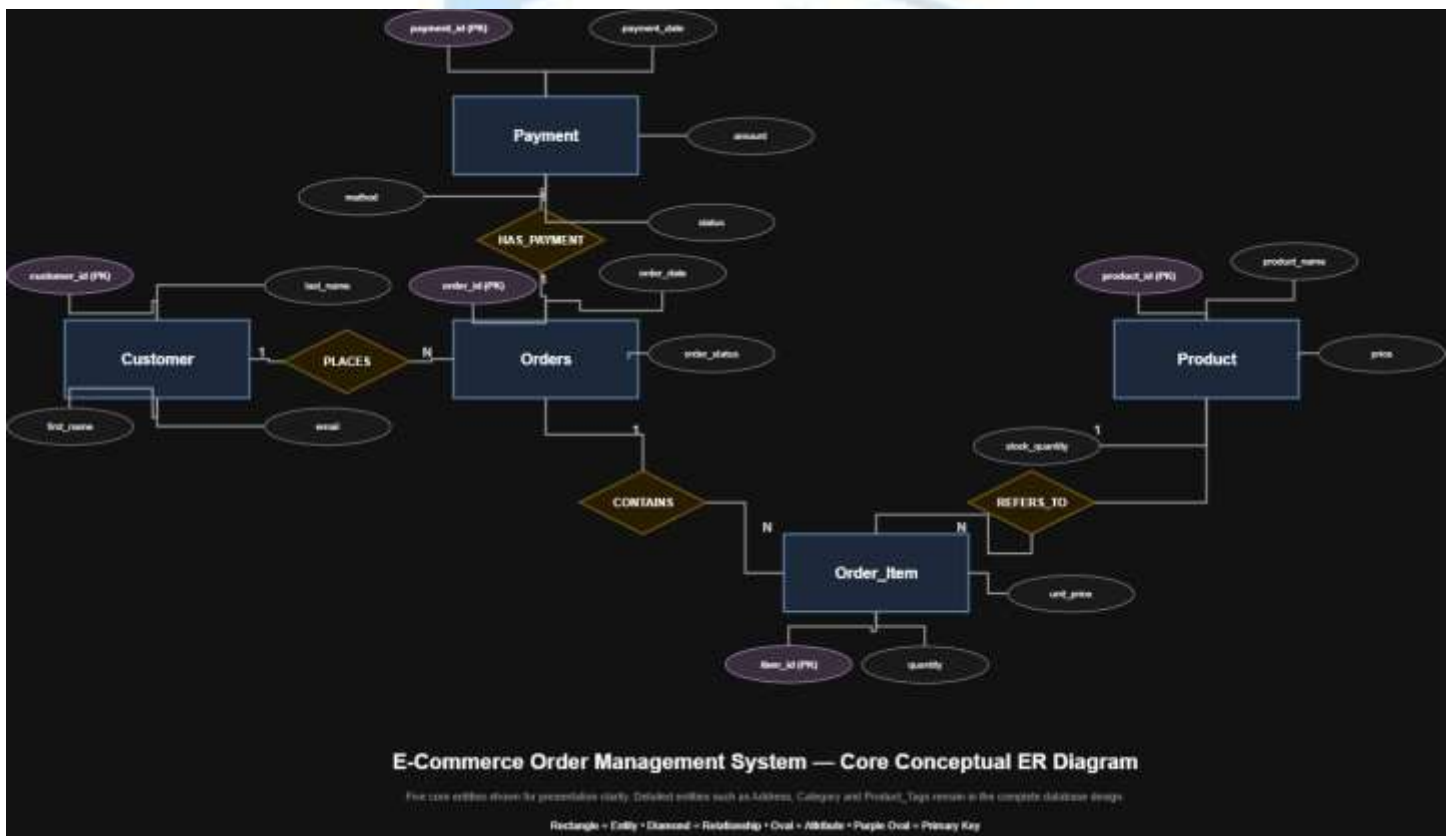
S.No	Entity Name	Description
1	Customer	Stores customer identity and registration details.
2	Customer_Phone	Stores customer phone numbers as a separate relation.
3	Address	Stores delivery addresses belonging to customers.
4	Category	Stores product categories and optional parent-category hierarchy.
5	Product	Stores product information, prices, stock, and category.
6	Product_Tags	Stores product tags as a separate multi-valued relation.
7	Orders	Stores order header information and status.
8	Order_Item	Stores products, quantities, and historical unit prices within an order.
9	Payment	Stores payment details and transaction status.

6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Customer	customer_id	INT	PK	Simple
	first_name	VARCHAR(50)	—	Simple
	last_name	VARCHAR(50)	—	Simple
	email	VARCHAR(100)	—	Simple
	registration_date	DATE	—	Simple
Customer_Phone	phone_id	INT	PK	Simple
	customer_id	INT	FK	Simple
	phone	VARCHAR(15)	—	Simple
Address	address_id	INT	PK	Simple
	customer_id	INT	FK	Simple
	street	VARCHAR(200)	—	Simple
	city	VARCHAR(100)	—	Simple
	state	VARCHAR(100)	—	Simple
	pincode	VARCHAR(10)	—	Simple
	address_type	VARCHAR(20)	—	Simple
Category	category_id	INT	PK	Simple
	category_name	VARCHAR(100)	—	Simple
	parent_category_id	INT	FK	Simple
Product	product_id	INT	PK	Simple
	product_name	VARCHAR(200)	—	Simple
	description	TEXT	—	Simple
	price	DECIMAL(10,2)	—	Simple
	stock_quantity	INT	—	Simple
	category_id	INT	FK	Simple
Product_Tags	product_id	INT	PK/FK	Simple
	tag	VARCHAR(50)	PK	Simple
Orders	order_id	INT	PK	Simple
	customer_id	INT	FK	Simple
	address_id	INT	FK	Simple
	order_date	DATE	—	Simple
	order_status	VARCHAR(30)	—	Simple
Order_Item	item_id	INT	PK	Simple
	order_id	INT	FK	Simple
	product_id	INT	FK	Simple

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
	quantity	INT	—	Simple
	unit_price	DECIMAL(10,2)	—	Simple
Payment	payment_id	INT	PK	Simple
	order_id	INT	FK	Simple
	payment_date	DATE	—	Simple
	amount	DECIMAL(10,2)	—	Simple
	method	VARCHAR(30)	—	Simple
	status	VARCHAR(20)	—	Simple
	transaction_id	VARCHAR(100)	—	Simple

6.3 ER Diagram



6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Customer	has	Customer_Phone	1:N
Customer	has	Address	1:N
Category	contains	Product	1:N
Category	has subcategory	Category	1:N
Product	has	Product_Tags	1:N
Customer	places	Orders	1:N
Address	is used by	Orders	1:N
Orders	contains	Order_Item	1:N
Product	appears in	Order_Item	1:N
Orders	has	Payment	1:1

6.5 Relational Database Schema

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Customer	customer_id	INT	PK	PK; email UNIQUE NOT NULL; registration_date DEFAULT
Customer	first_name	VARCHAR(50)	—	PK; email UNIQUE NOT NULL; registration_date DEFAULT
Customer	last_name	VARCHAR(50)	—	PK; email UNIQUE NOT NULL; registration_date DEFAULT
Customer	email	VARCHAR(100)	—	PK; email UNIQUE NOT NULL; registration_date DEFAULT
Customer	registration_date	DATE	—	PK; email UNIQUE NOT NULL; registration_date DEFAULT
Customer_Phone	phone_id	INT	PK	PK; FK customer_id; UNIQUE(customer_id,phone)
Customer_Phone	customer_id	INT	FK	PK; FK customer_id; UNIQUE(customer_id,phone)
Customer_Phone	phone	VARCHAR(15)	—	PK; FK customer_id; UNIQUE(customer_id,phone)
Address	address_id	INT	PK	PK; FK customer_id; NOT NULL address fields
Address	customer_id	INT	FK	PK; FK customer_id; NOT NULL address fields
Address	street	VARCHAR(200)	—	PK; FK customer_id; NOT NULL address fields
Address	city	VARCHAR(100)	—	PK; FK customer_id; NOT NULL address fields
Address	state	VARCHAR(100)	—	PK; FK customer_id; NOT NULL address fields
Address	pincode	VARCHAR(10)	—	PK; FK customer_id; NOT NULL address fields
Address	address_type	VARCHAR(20)	—	PK; FK customer_id; NOT NULL address fields
Category	category_id	INT	PK	PK; category_name UNIQUE; self-FK parent_category_id
Category	category_name	VARCHAR(100)	—	PK; category_name UNIQUE; self-FK parent_category_id
Category	parent_category_id	INT	FK	PK; category_name UNIQUE; self-FK parent_category_id
Product	product_id	INT	PK	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product	product_name	VARCHAR(200)	—	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product	description	TEXT	—	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product	price	DECIMAL(10,2)	—	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product	stock_quantity	INT	—	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product	category_id	INT	FK	PK; FK category_id; price CHECK >=0; stock CHECK >=0
Product_Tags	product_id	INT	PK/FK	Composite PK (product_id,tag); FK product_id
Product_Tags	tag	VARCHAR(50)	PK	Composite PK (product_id,tag); FK product_id

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Orders	order_id	INT	PK	PK; FK customer_id,address_id; status DEFAULT
Orders	customer_id	INT	FK	PK; FK customer_id,address_id; status DEFAULT
Orders	address_id	INT	FK	PK; FK customer_id,address_id; status DEFAULT
Orders	order_date	DATE	—	PK; FK customer_id,address_id; status DEFAULT
Orders	order_status	VARCHAR(30)	—	PK; FK customer_id,address_id; status DEFAULT
Order_Item	item_id	INT	PK	PK; FKs order_id,product_id; quantity CHECK >0; UNIQUE(order_id,product_id)
Order_Item	order_id	INT	FK	PK; FKs order_id,product_id; quantity CHECK >0; UNIQUE(order_id,product_id)
Order_Item	product_id	INT	FK	PK; FKs order_id,product_id; quantity CHECK >0; UNIQUE(order_id,product_id)
Order_Item	quantity	INT	—	PK; FKs order_id,product_id; quantity CHECK >0; UNIQUE(order_id,product_id)
Order_Item	unit_price	DECIMAL(10,2)	—	PK; FKs order_id,product_id; quantity CHECK >0; UNIQUE(order_id,product_id)
Payment	payment_id	INT	PK	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	order_id	INT	FK	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	payment_date	DATE	—	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	amount	DECIMAL(10,2)	—	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	method	VARCHAR(30)	—	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	status	VARCHAR(20)	—	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE
Payment	transaction_id	VARCHAR(100)	—	PK; order_id UNIQUE/FK; amount CHECK >=0; transaction_id UNIQUE

6.6 Normalization

UNF → 1NF → 2NF → 3NF

UNF(UnnormalizedForm):

Initially, customer information, multiple phone numbers, addresses, product details, multiple products in an order, and product tags may exist together in a single record. This can result in repeating groups and data redundancy.

1NF (First Normal Form):
All attributes are made atomic and repeating/multi-valued attributes are separated into individual tables.

- Customer phone numbers → Customer_Phone
- Product tags → Product_Tags
- Multiple products in an order → Order_Item
- Customer addresses → Address

Each table contains atomic values and each record is uniquely identified by a primary key.

2NF (Second Normal Form):
Partial dependencies are removed by separating attributes according to the entity they describe.

- Customer details are stored only in Customer.
- Product details are stored only in Product.
- Order-level details are stored in Orders.
- Product quantity and the price at the time of purchase are stored in Order_Item.
- Payment information is stored separately in Payment.

Thus, non-key attributes depend on the appropriate complete key/entity record.

3NF (Third Normal Form):
Transitive dependencies and unnecessary derived data are avoided.

- Category information is stored in Category rather than repeatedly in Product.
- Customer address information is stored in Address.
- Payment details are stored in Payment.
- Order totals are calculated from Order_Item rather than redundantly stored.
- Customer total orders are calculated from Orders rather than stored as a separate attribute.

Therefore, the database is organized up to **3NF**, reducing insertion, update, and deletion anomalies and minimizing data redundancy.

7. DATABASE IMPLEMENTATION

1. DDL — Database and Table Creation

```
-- =====  
-- 1. CUSTOMER  
-- =====  
CREATE TABLE Customer (  
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    registration_date DATE NOT NULL DEFAULT CURRENT_DATE  
);
```

```
-- =====  
-- 2. CUSTOMER PHONE  
-- =====  
CREATE TABLE Customer_Phone (  
    phone_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id INTEGER NOT NULL,  
    phone VARCHAR(15) NOT NULL,  
    UNIQUE (customer_id, phone),  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);
```

```
-- =====  
-- 3. ADDRESS  
-- =====  
CREATE TABLE Address (  
    address_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id INTEGER NOT NULL,  
    street VARCHAR(200) NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    state VARCHAR(100) NOT NULL,  
    pincode VARCHAR(10) NOT NULL,  
    address_type VARCHAR(20) NOT NULL DEFAULT 'Home',  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);
```

4. CATEGORY

```
CREATE TABLE Category (  
    category_id INT IDENTITY(1,1) PRIMARY KEY,  
    category_name VARCHAR(100) NOT NULL UNIQUE,  
    parent_category_id INT NULL,  
    FOREIGN KEY (parent_category_id) REFERENCES Category(category_id)
```

5. PRODUCT

```
CREATE TABLE Product (  
    product_id INT IDENTITY(1,1) PRIMARY KEY,  
    product_name VARCHAR(200) NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),  
    category_id INT NULL,  
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
```

- 6. PRODUCT TAGS

```
CREATE TABLE Product Tags (  
    product_id INT NOT NULL,  
    tag VARCHAR(50) NOT NULL,  
    PRIMARY KEY (product_id, tag),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
```

7. ORDERS

```
CREATE TABLE Orders (  
    order_id INT(11) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id INT(11) NOT NULL,  
    address_id INT(11) NOT NULL,  
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    order_status VARCHAR(30) NOT NULL DEFAULT 'Pending',  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
    FOREIGN KEY (address_id) REFERENCES Address(address_id)
```

- 8. ORDER ITEMS

```
CREATE TABLE Order Item (  
    item_id INT(11) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    order_id INT(11) NOT NULL,  
    product_id INT(11) NOT NULL,  
    quantity INT(11) NOT NULL CHECK (quantity > 0),  
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
    UNIQUE (order_id, product_id),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
```

- 9. PAYMENT

```
CREATE TABLE Payment (  
    payment_id INT(11) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    order_id INT(11) NOT NULL UNIQUE,  
    payment_date DATE,  
    amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),  
    merchant_id INT(11) { 0 } NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
    transaction_id VARCHAR(100) UNIQUE,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
```

7.2 Sample Data — DML

Table	No. of Records	Sample Data / Screenshot Reference
Customer	5	Sample customer records inserted; see SQL script.
Customer_Phone	5	One phone number per sample customer.
Address	5	Home/office delivery addresses.
Category	4	Electronics, Fashion, Mobile Accessories, Laptops.
Product	5	Sample products with prices and stock.
Product_Tags	8	Product tags stored separately.
Orders	5	Delivered, Shipped, Pending and Cancelled examples.
Order_Item	8	Order line items with quantity and unit price.
Payment	5	Success, Pending and Failed payment examples.

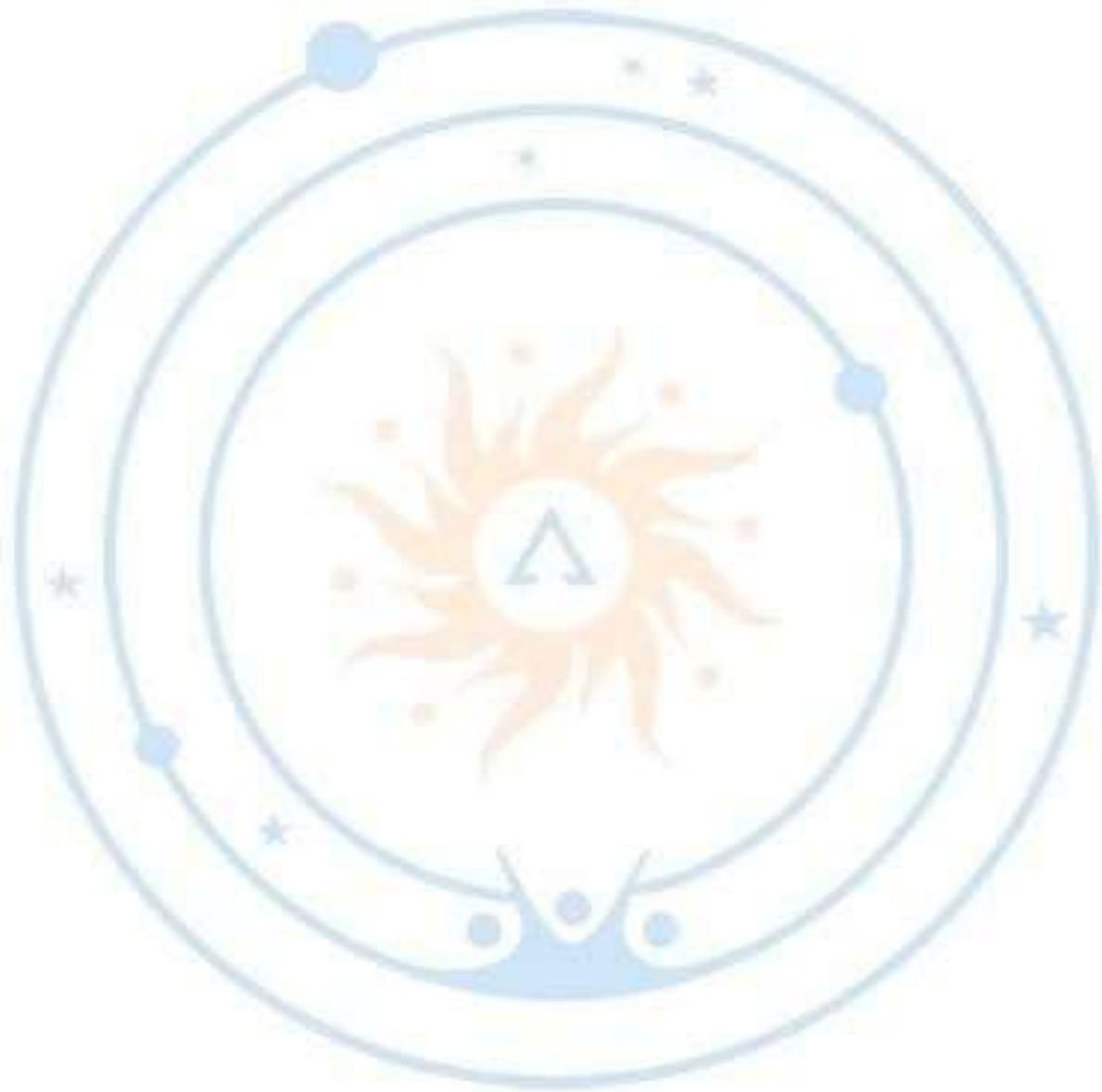
7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All master/transaction tables	Unique row identification.	SQL DDL / Table structure
FOREIGN KEY	Related tables	Maintains referential integrity and prevents orphan records.	SQL DDL
UNIQUE	Customer.email, Category.category_name, Payment.transaction_id	Prevents duplicate unique business values.	SQL DDL
NOT NULL	Mandatory customer, product, order and payment fields	Prevents missing required data.	SQL DDL
CHECK	Product.price, Product.stock_quantity, Order_Item.quantity/unit_price, Payment.amount	Rejects invalid numeric values.	SQL DDL
DEFAULT	Registration date, order date, order status, payment status	Provides valid default values.	SQL DDL
Trigger	Order_Item insert	Prevents an order item from exceeding available stock.	Trigger definition / execution

7.4 SQL Operations & Queries

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	SELECT * FROM Product;	Display all products.	SQL Script
2	Customer JOIN Orders	Display customers and their orders.	SQL Script
3	Order_Details view	Display complete order line details.	SQL Script
4	SUM(quantity × unit_price)	Calculate total sales excluding cancelled orders.	SQL Script
5	GROUP BY + ORDER BY	Find best-selling products.	SQL Script
6	GROUP BY + HAVING	Find customers spending more than ₹50,000.	SQL Script
7	WHERE stock_quantity < 50	Identify low-stock products.	SQL Script
8	Subquery with AVG(price)	Find products priced above average.	SQL Script
9	CREATE VIEW	Create reusable order/customer reports.	SQL Script
10	Stored procedure	Update an order status.	SQL Script
11	Trigger	Validate available stock before order item insertion.	SQL Script

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
12	CRUD	Demonstrate INSERT, SELECT, UPDATE and DELETE.	SQL Script



```
1 select * from product;
```

Results Chart

5 rows (Limited to only 100 rows)

Export ▾

product_id	product_name	description	price	stock_quantity	category_id
1	Samsung Galaxy S24	Android smartphone	65000.00	50	1
2	HP Pavilion Laptop	15-inch performance laptop	55000.00	30	4
3	Cotton T-Shirt	Comfortable cotton t-shirt	499.00	200	2
4	Wireless Earbuds	Bluetooth wireless earbuds	1999.00	100	3
5	Laptop Backpack	Water-resistant backpack	1499.00	75	4

Table Editor

schema public

+ New table

Search tables...

address UNRESTRICTED

category UNRESTRICTED

customer UNRESTRICTED

customer_order_s... UNRESTRICTED

customer_phone UNRESTRICTED

order_details UNRESTRICTED

order_item UNRESTRICTED

orders UNRESTRICTED

payment UNRESTRICTED

product UNRESTRICTED

product_tags UNRESTRICTED

address

+

Filter by address_id, customer_id, street.

Sort

RLS disabled

Role postgres

Insert

	address_id	customer_id	street varchar	city varchar
	1	1	12 MG Road	Vijayawada
	2	2	45 Park Street	Hyderabad
	3	3	7 Lake View	Visakhapatnam
	4	4	21 Main Road	Rajahmundry
	5	5	8 Green Avenue	Guntur



Figure 3: JOIN / order-details query and output



```
1 select * from order_details;
```

Results Chart							
8 rows (Limited to only 100 rows) Export							
order_id	customer_name	product_name	quantity	unit_price	subtotal	order_date	
1	Anita Patel	Samsung Galaxy S24	1	65000.00	65000.00	2024-08-01	
1	Anita Patel	Wireless Earbuds	1	1999.00	1999.00	2024-08-01	
2	Ramesh Nair	HP Pavilion Laptop	1	55000.00	55000.00	2024-08-05	
2	Ramesh Nair	Laptop Backpack	1	1499.00	1499.00	2024-08-05	
3	Anita Patel	Cotton T-Shirt	3	499.00	1497.00	2024-08-10	
4	Divya Singh	Cotton T-Shirt	2	499.00	998.00	2024-08-12	

```

1 SELECT p.product_name, SUM(oi.quantity) AS units_sold
2 FROM Product p
3 JOIN Order_Item oi ON p.product_id = oi.product_id
4 JOIN Orders o ON oi.order_id = o.order_id
5 WHERE o.order_status <> 'Cancelled'
6 GROUP BY p.product_id, p.product_name
7 ORDER BY units_sold DESC;
8 |

```

Results Chart

5 rows (Limited to only 100 rows)

Export ▾

product_name	units_sold
Cotton T-Shirt	5
Wireless Earbuds	3
Laptop Backpack	1
HP Pavilion Laptop	1
Samsung Galaxy S24	1

8. MODULES & FUNCTIONALITIES.

S.No	Module Name	Functionality
1	Customer Management	Stores customer details, registration date, and phone numbers.
2	Address Management	Maintains multiple customer delivery addresses.
3	Product & Category Management	Stores categories, products, prices, stock, and tags.
4	Order Management	Creates and tracks orders and their line items.
5	Payment Management	Records payment amount, method, transaction status, and transaction reference.
6	Reporting & Analytics	Generates order details, sales totals, best-selling products, low-stock lists, and customer spending reports.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: <https://github.com/E-commerce-Order-management-system/Smart-E-Commerce-Order-Management-System>

Student Name	Roll Number	Role / Responsibility	Contribution Status
A VENKATA NAVEEN	25B11AI075	Documentation, GitHub Repository	Completed
N KRANTHI	25B11AI606	SQL Implementation / Queries	Completed
N PHANINDRA	25B11AI839	ER Daigram, relationships	Completed
G KISHORE	25B11AIA79	Testing / PPT / Presentation	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Future improvements may include a customer/admin web interface, authentication and role-based access control, a dedicated shipment/delivery module, inventory reservation during checkout, automated invoice generation, indexing for high-volume order searches, transaction handling for multi-step order placement, and dashboard-style sales reporting.

11. CONCLUSION

The E-Commerce Order Management System demonstrates how a relational database can organize the core information required for an online store. The project models customers, addresses, categories, products, order items, orders, and payments using primary and foreign keys and integrity constraints. Normalization separates repeating and multi-valued information and keeps transaction facts consistent. SQL DDL, DML, DQL, CRUD operations, joins, aggregate queries, GROUP BY/HAVING, subqueries, views, a stored procedure, and a stock-validation trigger provide a practical implementation of DBMS concepts. The completed database can be executed in pgAdmin 4 and used to demonstrate order processing and reporting.
